

JavaAgent CLI — 项目报告

生成时间: 2026-06-07 | Commit: `d51f04c` | Branch: `main`

一、项目概览

JavaAgent CLI 是一个基于 **ReAct (Reasoning + Acting)** 架构的自主编程智能体，使用 Java 21 实现。它将大语言模型与本地开发工具链深度集成，通过 Tool-Augmented LLM 模式实现文件读写、代码编辑、Shell 执行、HTTP 请求等能力，参照 Claude Code 的交互范式设计。

属性	值
----- -----	
语言	Java 21 (records、pattern matching、sealed interfaces)
构建	Maven + maven-shade-plugin (fat JAR)
终端	JLine3 (行编辑、历史翻页、Tab 补全)
模型	OpenAI 兼容 API + SSE 流式 + reasoning_content 思维链
测试	JUnit 5 + MockModelClient 离线测试
跨平台	Windows (PowerShell/cmd.exe) / macOS / Linux

二、代码统计

2.1 文件与行数

类别	文件数	代码行数	平均行数/文件
----- ----- ----- -----			
源码 (main)	42	6,143	146
测试 (test)	13	1,277	98
合计	**55**	**7,420**	**135**

2.2 测试覆盖

指标	数值
----- -----	
测试类	13 个
测试方法	66 个
通过率	100%

测试覆盖的核心模块：

测试类	覆盖范围	测试数
----- ----- -----		
`AgentTest`	ReAct 循环、迭代上限、连续失败中断、compact 压缩	5
`ReadFileToolTest`	文件读取、分页、二进制检测	5
`WriteFileToolTest`	文件写入、路径校验、大小限制	5
`EditToolTest`	精确替换、多处匹配、diff 生成	5
`GrepToolTest`	正则搜索、大小写、路径过滤	4
`ListDirectoryToolTest`	目录列表、递归、工作区限制	4
`BashToolTest`	命令执行、超时、危险命令检测	6
`NetworkToolTest`	HTTP 请求、方法、超时	4
`ApprovalManagerTest`	策略链、审批缓存、bypass 模式	6
`ConfigTest`	配置加载、默认值、热重载	5
`ConversationManagerTest`	会话持久化、加载、列表	5
`IntegrationTest`	端到端 ReAct 流程	4
`TokenCounterTest`	jtokit token 计数	5

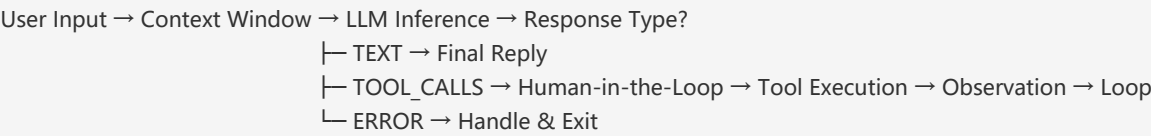
2.3 依赖清单

依赖	版本	用途
----- ----- -----		
`com.fasterxml.jackson.core:jackson-databind`	2.17.0	JSON 序列化/反序列化
`org.jline:jline`	3.26.3	终端行编辑、Tab 补全
`com.knuddels:jtokkit`	1.1.0	tiktoken Java 移植 (cl100k_base 编码)
`org.junit.jupiter:junit-jupiter`	5.10.3	单元测试框架

三、架构设计

3.1 ReAct 推理循环

核心流程（`Agent.java`）：



关键设计决策：

机制	说明
----- -----	
ReAct Loop	推理-行动交替循环，最多 12 轮（可配置），支持多步工具编排
Human-in-the-Loop	只读操作自动放行，写操作需用户显式批准，外部路径默认拒绝

Streaming Inference	SSE 逐 token 流式输出，支持 `reasoning_content` 思维链
Effort Control	4 级推理深度 (low/medium/high/max) ，通过 System Prompt 注入控制
Failure Recovery	同一工具连续失败 3 次自动中断，防止无限循环
Guardrails	Unix 10 类 + Windows 10 类危险命令正则检测、工作区沙箱隔离、敏感信息过滤

3.2 上下文管理

本项目实现了两级上下文管理策略：

- **Token 级可视化****（`/context` 命令）：
- 使用 jtokkit (tiktoken Java 移植) 进行精确 token 计数
 - 按 System Prompt / Tool Definitions / Messages 三类分项统计
 - 彩色柱状图实时显示占用比例（■ 实心 / □ 空心）
- **LLM 摘要压缩****（`/compact` 命令 + 自动触发）：
- 达到阈值（默认 80%）时自动触发，或手动 `/compact`
 - 保留最近 6 条消息 + LLM 生成的中文摘要
 - 转圈动画（Braille spinner）反馈压缩进度

3.3 工具系统

8 个内置工具 + SPI 插件扩展：

工具	别名	权限	说明
----- ----- ----- -----			
`read_file`	`cat`	只读	读取文本文件（256KB 上限，自动分页）
`write_file`	`write`, `save_file`	读写	写入文件（带 60 行预览，100KB 上限）
`edit`	`replace`, `sed`	读写	精确字符串替换（带彩色 diff 预览）
`delete_file`	`rm`	读写	删除文件（禁止删除工作区根目录）
`grep`	`search`, `find`	只读	正则搜索（跳过 target/.git，最多 100 目录）
`list_directory`	`ls`, `dir`	只读	递归列出目录内容
`bash`	`shell`, `exec`	读写	Shell 命令（默认禁用，10s 超时）
`network`	`http`, `fetch`	读写	HTTP 请求

插件扩展：实现 `ToolProvider` 接口 → `META-INF/services/com.javagent.tools.ToolProvider` → `ServiceLoader` 自动发现。

3.4 安全机制

机制	实现
----- -----	
工作区隔离	所有文件操作限制在项目目录内
审批确认	写文件、删文件、bash、网络请求需要用户确认
危险命令检测	Unix 10 类 + Windows 10 类正则匹配

敏感信息脱敏	API Key/Token 正则过滤，防止泄漏到会话文件
连续失败保护	同一工具连续失败 3 次自动中断

Windows

危险命令覆盖: `del/erase`、`rd/rmdir`、`Remove-Item`、`format`、`shutdown/restart`、`Stop-Computer`、`taskkill`、`bcdedit`、`Set-ExecutionPolicy`、`cipher`、`takeown`、`icacls`。

四、项目结构

```
src/main/java/com/javagent/
├─ JavaAgentCLI.java      # 1014 行 — CLI 入口、REPL、Slash Command 路由、Spinner
├─ BannerPrinter.java     # 启动横幅、运行时状态栏
├─ SlashCommandCompleter.java # JLine3 Tab 补全
├─ core/
│  ├─ Agent.java         # 358 行 — ReAct 核心循环引擎、compact 压缩
│  ├─ Config.java        # Properties 配置管理 (热重载)
│  ├─ ConversationManager.java # 多会话 JSON 持久化
│  ├─ ApprovalManager.java # 权限策略引擎 + 审批缓存
│  ├─ ContextUsage.java  # 上下文 token 使用量数据模型
│  └─ ToolStats.java     # 工具调用指标 (次数/耗时/错误率)
├─ model/
│  ├─ ModelClient.java   # 模型客户端抽象接口
│  ├─ OpenAiCompatibleModelClient.java # SSE 流式 + 重试 + 限流
│  ├─ MockModelClient.java # 关键词匹配 Mock (离线调试)
│  ├─ Message.java       # 消息记录 (含 reasoningContent)
│  ├─ ToolCall.java      # 工具调用记录
│  ├─ ModelResponse.java  # 模型响应 (TEXT/TOOL_CALLS/ERROR)
│  └─ TextStreamHandler.java # 流式输出回调接口
├─ tools/
│  ├─ Tool.java          # 工具抽象接口
│  ├─ ToolDefinition.java # 工具定义 (名称、描述、参数 Schema)
│  ├─ ToolExecutionResult.java # 执行结果 (content + error)
│  ├─ ToolRegistry.java  # 名称/别名查找 + SPI 插件发现
│  ├─ ToolProvider.java  # SPI 扩展接口
│  ├─ FileToolSupport.java # 文件工具公共逻辑
│  ├─ ReadFileTool.java  # 读文件
│  ├─ WriteFileTool.java # 写文件
│  ├─ EditTool.java      # 精确文本替换
│  ├─ DeleteFileTool.java # 删除文件
│  ├─ GrepTool.java      # 正则搜索
│  ├─ ListDirectoryTool.java # 目录列表
│  ├─ BashTool.java      # Shell 命令 (跨平台)
│  └─ NetworkTool.java   # HTTP 请求
└─ util/
   ├─ Terminal.java      # ANSI 渲染 + splitLines 跨平台换行
   ├─ TokenCounter.java  # jtokkit token 计数
   ├─ ContextDisplay.java # /context 彩色柱状图
   ├─ MarkdownRenderer.java # Markdown → ANSI
   ├─ Sanitizer.java     # 敏感信息脱敏
   └─ RateLimiter.java   # 令牌桶限流
```

五、斜杠命令

命令				说明		
----- -----						
`/help`				显示帮助信息		
`/exit` `/quit`				退出程序		
`/clear` `/new [title]`				新建会话		
`/save [title]`				保存当前会话		
`/load [id\`		title\`		latest\`	加载已保存的会话	
`/sessions`				列出已保存的会话		
`/tools`				列出已注册的工具		
`/mode mock\`		real\`			切换模型模式	
`/stream on\`		off\`			开关流式输出	
`/bash on\`		off\`			开关 Bash 工具	
`/bypass on\`		off\`			跳过所有工具审批确认	
`/effort low\`	medium\`		high\`		max\`	调节模型推理深度
`/context`				查看上下文 token 占用（彩色柱状图）		
`/compact`				压缩对话历史（LLM 摘要，节省 token）		
`/prompt show\`		set\`		reset\`	管理自定义 System Prompt	
`/approvals clear`				清空审批缓存		
`/reload`				重新加载配置文件		
`/export`				导出当前对话为 Markdown		
`/stats`				查看工具调用统计		
`/status`				显示运行时状态		

六、配置参考

在项目根目录创建 `config.properties`（以下为默认值）：

```
# 模型
agent.mock_mode=true
agent.api_key=
agent.base_url=https://api.openai.com/v1
agent.model=gpt-5.4-mini
agent.effort=medium      # 推理深度: low | medium | high | max

# 行为
agent.auto_save=true
agent.max_iterations=12  # ReAct 最大循环次数
agent.enable_bash=false
agent.stream_responses=true
agent.approval_cache=true
agent.allow_external_paths=false
agent.bypass_permissions=false

# 上下文
agent.system_prompt=      # 自定义 System Prompt
agent.max_tokens=200000   # 模型上下文窗口 token 数
agent.compact_threshold=0.8 # 自动压缩阈值 (0.0-1.0)
agent.rate_limit_qps=10    # API 请求限流 (QPS)
```

配置文件查找顺序: 项目根目录 → 工作目录 → `~/javaagent-cli/config.properties`。

七、构建与运行

```
# 环境要求: Java 21+、Maven 3.8+

# 构建
mvn clean package -DskipTests

# 运行测试
mvn test

# 运行 (Linux / macOS)
javaagentcli

# 运行 (Windows)
javaagentcli.cmd

# 运行 (直接)
java --enable-native-access=ALL-UNNAMED -jar target/javaagent-cli-1.0.0.jar

# 运行 (Mock 模式, 无需 API key)
java -jar target/javaagent-cli-1.0.0.jar --mock
```

八、Git 历史

d51f04c feat: token-level context management, cross-platform fixes, compact with spinner
7cfa222 Merge pull request #1 from cuac333/feat/cli-enhancements
2408912 feat: suppress warnings, slash command hints, dynamic status, /context & /effort commands
66c2892 Enhance system prompt with clarification and action distinction guidelines
43f6adc feat: major upgrade — EditTool, SSE streaming, Claude Code UI, security hardening
a1d0600 Initial JavaAgent CLI project

九、关键源码

9.1 Agent 核心 ReAct 循环

```
// Agent.java — processTurn 核心循环 (节选)
for (int i = 0; i < config.maxIterations(); i++) {
    ModelResponse response = modelClient.chat(systemPrompt, freshContext,
        toolRegistry.definitions(), effectiveStreamHandler);

    if (response.isText()) {
        conversationManager.addAssistantMessage(response.content(), response.reasoningContent());
        return response.content();
    }

    conversationManager.addAssistantToolCallMessage(response.content(), response.toolCalls(), ...);
    for (ToolCall toolCall : response.toolCalls()) {
        ToolExecutionResult result = executeToolCall(toolCall, approvalHandler);
        // 连续失败 3 次 → 中断
        if (result.error() && toolCall.name().equals(lastFailedTool)) {
            if (++consecutiveFailures >= 3) return stuckText;
        } else { consecutiveFailures = 0; }
        // 脱敏后存入对话历史
        conversationManager.addToolResultMessage(toolCall.id(), toolCall.name(),
            Sanitizer.sanitize(result.content()), result.error());
    }
}
```

9.2 Compact 压缩与 Spinner 动画

```
// JavaAgentCLI.java — /compact 命令 (节选)
case "/compact" -> {
    Spinner compactSpinner = new Spinner(out, new AtomicBoolean(false), "压缩中...");
    compactSpinner.start();
    try {
        String result = agent.compact();
        out.println(green(" ") + result);
    } catch (Exception e) {
        out.println(red(" ") + "压缩失败: " + e.getMessage());
    } finally {
        compactSpinner.stop();
    }
}

// Agent.java — compact() 方法 (节选)
public String compact() {
    List<Message> context = conversationManager.currentContext();
    if (context.size() < COMPACT_MIN_MESSAGES) return "消息数量不足, 无需压缩.";
    // 保留最近 6 条, 其余发给 LLM 做摘要
    int keepEnd = Math.min(COMPACT_KEEP_RECENT, context.size() - 1);
    List<Message> toSummarize = context.subList(0, context.size() - keepEnd);
    List<Message> toKeep = context.subList(context.size() - keepEnd, context.size());
    ModelResponse response = modelClient.chat(COMPACT_SYSTEM_PROMPT, summaryRequest, List.of(), null);
    // 摘要 + 最近消息 → 替换对话历史
    List<Message> newMessages = new ArrayList<>();
    newMessages.add(Message.system("[对话摘要] " + response.content()));
    newMessages.addAll(toKeep);
    conversationManager.replaceMessages(newMessages);
    return "压缩完成: 保留 " + toKeep.size() + " 条最近消息 + 摘要";
}
```

报告生成: Claude Code (claude.ai/code)